FRONT-END PROJECT DESCRIPTION · APPLICATION DOCUMENT

Putting radiotherapy dose data in the doctor's hands.

A cross-platform Flutter application that lets radiation oncologists visualize DICOM-RT data (CT, structures and 3D dose) and adjust dose constraint values for the heart. Built on a Node.js middleware layer and the Orthanc DICOM server.

Flutter · Dart | Node.js | Orthanc | DICOMweb | DICOM-RT (RTSTRUCT · RTDOSE · RTPLAN) | Healthcare

IN THIS DOCUMENT

01 The problem: plan review was tied to the workstation

02 System architecture: Flutter, Node.js, Orthanc

03 The front end in detail: rendering, interaction, performance

04 My contribution and the hard problems

05 Outcome and reflection

Marco Casanova

Mobile Developer

Munich, Germany

marco.casanova.de@outlook.com

marcocasanova.de

July 2026

01 · CONTEXT

The problem: plan review was tied to the workstation

In radiotherapy every treatment plan is a compromise: enough dose to control the tumor, and as little as possible to the healthy tissue around it. For treatments in the chest area the **heart is one of the most important organs at risk**, because the dose it receives is linked to cardiac problems years later. Doctors therefore check plans against heart dose constraints such as the mean heart dose or the volume of the heart that receives a certain dose level.

The data behind this decision is standard DICOM-RT: the planning **CT series**, the **RTSTRUCT** with the organ contours, the **RTDOSE** grid and the **RTPLAN**. In practice, all of it was locked inside dedicated planning workstations. When a doctor wanted to discuss whether the heart could be spared further, the conversation usually happened away from the data, in a hallway or a tumor board, based on memory or printed screenshots. Changing a constraint and seeing its effect required a workstation, a physicist and time.

> **The product idea:** a tablet app where a doctor opens a patient's plan, sees the CT with the organ contours and the dose overlay, checks the dose-volume curves and **adjusts the constraint values for the heart**. The app shows immediately whether the current plan satisfies them, and the doctor sends that feedback to the planning team.

One boundary we drew early: this is a **review and decision-support tool, not a treatment planning system**. Doctors adjust constraint values and write annotations, they do not modify dose. That kept the clinical scope honest and the regulatory footprint manageable. It also let us focus the front end on the one thing it had to do well: make complex volumetric data readable and interactive on a device you can carry into a meeting.

My role

I shaped the front end and the integration layer behind it. I proposed Flutter and argued for it, designed the visualization and interaction architecture of the app, defined the JSON contract between client and server, and built the Node.js middleware that connects the app to Orthanc. The rest of this document describes the system, the front-end work in detail, and the problems that turned out to be hard.

Who it serves

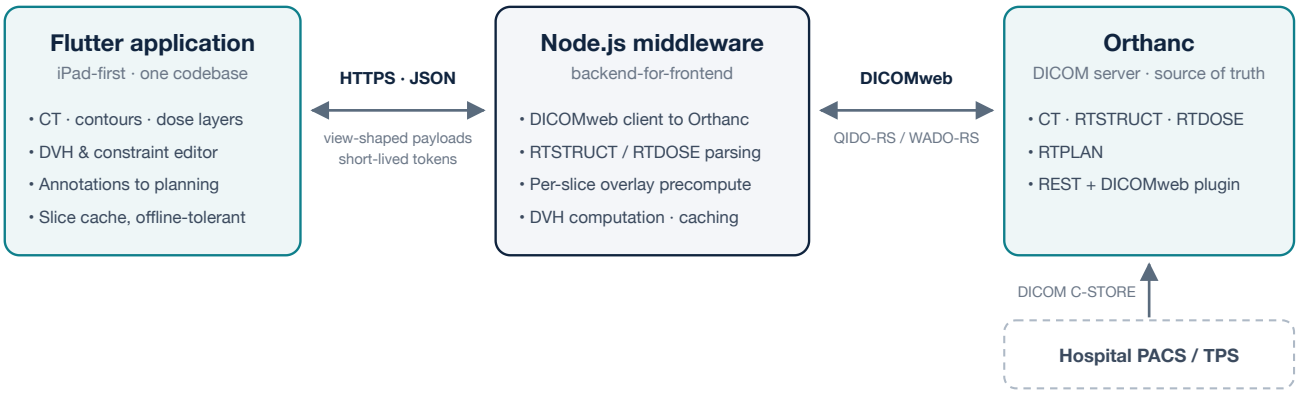
Radiation oncologists reviewing plans, and the planning team (physicists, dosimetrists), who get structured feedback tied to the data instead of verbal notes.

What changed

Constraint discussions moved from the workstation room to wherever the doctor happens to be, with the actual dose distribution and DVH curves on screen.

02 · SYSTEM ARCHITECTURE

Three layers, each doing what it's good at



> The Flutter client never speaks DICOM. It consumes a purpose-built JSON contract. The Node.js layer handles all DICOM specifics and talks DICOMweb to Orthanc, which receives the studies from the hospital systems.

Orthanc: the imaging source of truth

Orthanc is a lightweight open source DICOM server. It receives the planning studies from the hospital systems over standard DICOM and exposes them over DICOMweb. Using a standards-compliant server meant the product plugged into existing clinical infrastructure instead of asking the clinic to change its setup.

Node.js: the layer that protects the client

The middleware exists for one main reason: **a tablet should not parse a 3D dose grid**. RT objects are large, irregular and full of vendor quirks. The Node layer queries and retrieves studies via QIDO-RS and WADO-RS, parses the RTSTRUCT contours and the RTDOSE grid once, precomputes per-slice dose overlays and DVH curves on the server, caches the results, and hands the app compact JSON and image payloads shaped for the views that need them. It also handles authentication and exchanges the hospital identity for short-lived tokens, so credentials never sit on a device.

Flutter: one rendering canvas, every screen

The front end is a single Dart codebase, built tablet-first. The main argument for Flutter was not code sharing but **control over rendering**: this kind of visualization is mostly custom drawing, and Flutter gives you one predictable painting pipeline across iPad, desktop and web, with no per-platform differences in how a contour or a dose overlay is drawn.

What this shape bought us

App releases stayed independent from hospital integration work. Vendor quirks are handled and tested in the middleware, behind a contract the client can rely on.

Clear interfaces at every boundary

DICOM at the hospital boundary, DICOMweb between the server layers, plain JSON towards the client. Each interface uses the simplest option that works reliably.

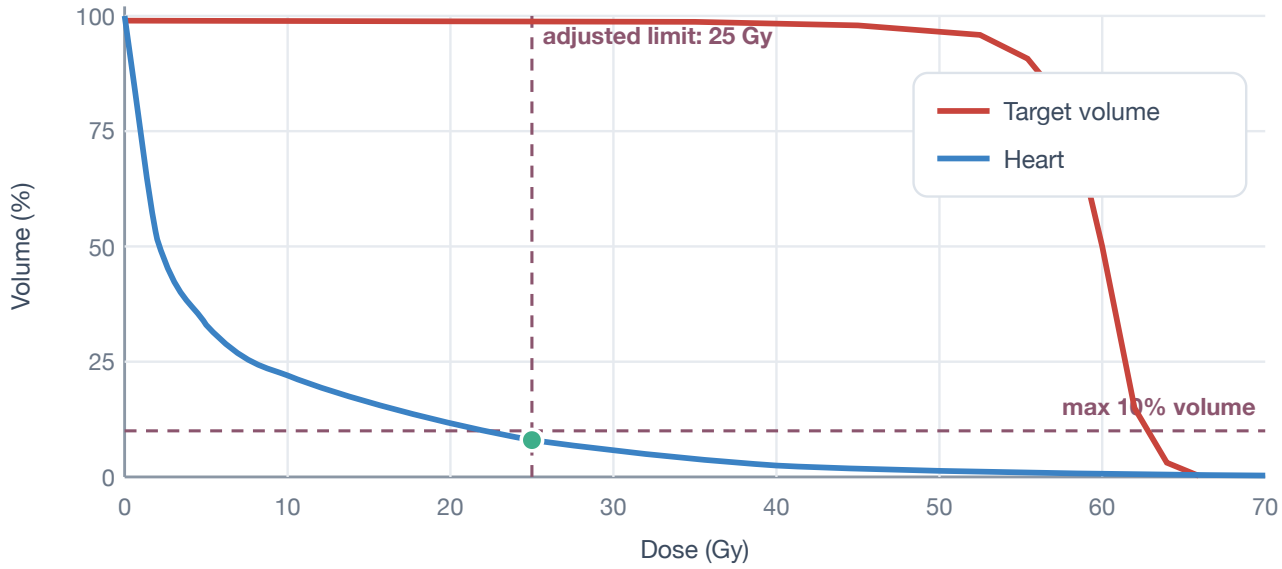
Rendering DICOM-RT data on a tablet

A layered painting pipeline

The viewer is a stack of custom-painted layers on one canvas: the windowed **CT slice** at the bottom, the **dose overlay** above it with adjustable opacity, the **organ contours** from the RTSTRUCT as stroked paths (the heart always visually prominent), and the constraint badges on top. All geometry lives in **patient space**, in millimetres, and a single transform maps it to the screen. That one rule keeps CT pixels, dose voxels and contour lines aligned while the user zooms, pans and scrubs through slices.

Interaction model

Doctors scrub through slices with a swipe, adjust the CT window with gestures or presets, toggle structures on and off, and change the dose opacity. The central interaction is the **constraint editor**: the heart's dose metrics are shown next to editable constraint values, and when the doctor changes a value, the DVH panel and the slice badges update right away. The plan either satisfies the new constraint or visibly does not. A question like "what if we demanded more heart sparing?" gets answered on screen in a few seconds, and the decision is stored as an annotation for the planning team.



Illustrative dose-volume histogram (DVH) as shown in the constraint editor. The doctor tightens the heart limit to 25 Gy at max 10% volume, and the heart curve is checked against it live (green marker: satisfied).

Performance engineering

- **Nothing heavy on the UI thread.** Slice decoding and geometry preparation run in Dart isolates. The raster thread only paints prepared objects.
- **Per-slice payloads instead of volumes.** The app requests exactly the slice images and precomputed dose overlays it needs. An LRU cache plus prefetching of neighbor slices keeps scrubbing smooth.
- **Paths built once.** Contour polylines become cached Path objects per slice, so pan and zoom are just a matrix change.

04 · MY CONTRIBUTION

My contribution and the hard problems

Ownership

- **Framework decision.** I proposed Flutter over native and web alternatives and defended the choice on rendering grounds: this product is custom canvas drawing first and widgets second, and one consistent painting pipeline across devices is easier to keep correct than three different ones.
- **Visualization architecture.** The layered painter stack, the patient-space coordinate model and the caching and prefetch strategy are designs I authored and iterated with the team.
- **The client-server contract.** I defined the JSON API the app consumes and built the Node.js middleware behind it, including the DICOMweb integration with Orthanc and the server-side RT processing.
- **Clinical UX iteration.** I sat with doctors, watched them use early builds, and reworked the constraint editor until changing a heart limit felt like adjusting a number instead of operating software.

The hard problems

1 · Big data, small device

A dose grid plus a CT series is far more than a tablet should hold or parse. The fix was architectural: move parsing and precomputation to the server, define per-slice payloads, and let the client focus on caching and painting.

2 · Geometric correctness

A contour drawn one voxel off is not a cosmetic bug, it is clinically wrong. We made patient space the single geometric authority, derived every mapping from the DICOM spatial attributes, and compared our rendering against reference output from the planning system on known datasets before trusting it.

3 · Earning clinical trust

Doctors will not use a viewer they cannot trust at a glance. Orientation markers, units and slice position are always visible, the window presets match what clinicians already know, and the dose overlay uses a conventional colormap instead of a novel one.

4 · Scope as a safety boundary

The most important decision was what the app refuses to do: it never modifies dose. Doctors adjust constraint values and annotate, planning stays in the planning system. That kept the product safe and its purpose clear to everyone involved.

One thing I keep from this project: in clinical software the front end is not just a presentation layer, it is part of the safety surface. Most of the work that mattered was about making sure the picture on screen could be trusted.

What changed, and what I'd do differently

Outcome

Plan review stopped being tied to a workstation. Doctors could open a plan on a tablet, look at the dose around the heart, tighten a constraint and see right away whether the plan still held, then hand the planning team an annotation tied to the data instead of a verbal note. The Node.js layer absorbed the DICOM vendor quirks we ran into over time, so the app's contract and release cycle stayed stable while the integration surface changed underneath.

What I would do differently

- **Build the geometric test harness first.** Comparing our rendering against reference datasets caught real deviations, and every one found late would have been cheaper to find on day one.
- **Treat DICOM quirks as fixtures from the start.** Each vendor peculiarity eventually became a test fixture in the middleware. Starting with that discipline would have saved several rounds of "works with this TPS but not that one".
- **Prototype the clinical interaction earlier.** The constraint editor only became good after watching doctors use it. The first sketches were built around the engineering, not around the decision.

Stack summary

Front end	Flutter (Dart): custom canvas rendering pipeline, isolate-based decoding, LRU slice caching, tablet-first UX
Middleware	Node.js backend-for-frontend: DICOMweb client, RTSTRUCT/RTDOSE parsing, server-side DVH and overlay precomputation, auth, caching
Imaging backend	Orthanc DICOM server: DICOMweb (QIDO-RS / WADO-RS), fed from hospital PACS/TPS over DICOM
Data	DICOM CT series · RTSTRUCT · RTDOSE · RTPLAN
Domain	Radiotherapy plan review: heart as organ at risk, dose constraint adjustment, clinical annotations

> Company, product and patient-related details are left out of this document on purpose. I am happy to walk through the system, including code-level decisions and the clinical iteration process, in as much depth as a conversation allows.

Marco Casanova
Mobile Developer · Munich, Germany
marco.casanova.de@outlook.com
marcocasanova.de